

MUSTAFACODE

Lecture 19

Overloading Stream Insertion & Extraction Operators in C++
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 19: Overloading Stream Insertion & Extraction Operators in C++

Lecture No. 19 — Overloading Stream Insertion & Extraction Operators in C++

Introduction

C++ mein hum data ko screen par display karne aur user se input lene ke liye streams use karte hain.

Do important operators hain:

Operator	Name	Purpose
<<	Stream Insertion Operator	Output display karna
>>	Stream Extraction Operator	Input lena

19.1 Stream Insertion Operator (<<)

Ye operator data ko output stream mein bhejne ke liye use hota hai.

Default output stream:

```
cout
```

Ye console screen par output show karta hai.

Simple Example

```
#include <iostream>
using namespace std;

int main() {

    int i = 1;
    int j = 2;

    cout << "i = " << i << endl;
    cout << "j = " << j << endl;

    return 0;
}
```

Output

```
i = 1
j = 2
```

Real-Life Example

Socho `cout` ek delivery boy hai.

Aur `<<` operator parcel bhejne ka kaam karta hai.

```
cout << data;
```

Matlab:

"Ye data screen tak pohcha do."

19.2 Stream Extraction Operator (<<)

Ye operator input stream se data lene ke liye use hota hai.

Default input stream:

```
cin
```

Simple Example

```
#include <iostream>
using namespace std;

int main() {

    int age;

    cout << "Enter your age: ";

    cin >> age;

    cout << "Your age is: " << age;

    return 0;
}
```

Output

```
Enter your age: 20
Your age is: 20
```

Real-Life Example

Socho `cin` ek collector hai.

Aur `>>` operator user se data lekar variable mein store karta hai.

```
cin >> age;
```

Matlab:

"User se value lo aur age mein store kar do."

Behind the Scene Working

Jab hum likhte hain:

```
cout << i;
```

Asal mein compiler ye function call karta hai:

```
operator<<(cout, i);
```

Similarly

```
cin >> i;
```

Actually:

```
operator>>(cin, i);
```

Cascading Statements

C++ multiple outputs/input ek hi line mein allow karta hai.

Example

```
cout << "Hello " << "World";
```

Input Example

```
cin >> a >> b;
```

Why Cascading Works?

Kyuki operators stream ka reference return karte hain.

User Defined Classes Problem

Basic data types ke liye operators already overloaded hote hain.

Lekin user-defined classes ke liye nahi.

Example

```
class Complex {  
  
};
```

```
Complex c1;  
  
cout << c1;
```

Compiler Error

Compiler nahi samjhega ke `Complex` object ko print kaise karna hai.

Isliye hume manually operator overload karna padega.

Complex Number Class

Complex number:

```
(Real Part, Imaginary Part)
```

Example:

```
(3, 4)
```

Yahan:

- 3 → Real Part
- 4 → Imaginary Part

19.3 Overloading Stream Insertion Operator (<<)

Wrong Approach (Member Function)

```
class Complex {  
  
public:  
  
    void operator<<(ostream&);  
};
```

Problem

Use kuch aise hoga:

```
c1 << cout;
```

Ye confusing syntax hai.

Aur cascading bhi work nahi karegi.

Correct Approach → Friend Function

Stream operators ko friend function banana best practice hai.

Complex Class

```
#include <iostream>
using namespace std;

class Complex {

private:

    float real;
    float img;

public:

    Complex(float r = 0, float i = 0) {

        real = r;
        img = i;
    }

    friend ostream& operator<<(ostream& os, const Complex& c);
};
```

Stream Insertion Operator Code

```
ostream& operator<<(ostream& os, const Complex& c) {

    os << "(" << c.real << ", " << c.img << ")";

    return os;
}
```

Explanation

`ostream& os`

Output stream object receive karega.

Usually:

```
cout
```

`const Complex& c`

Complex object receive karega.

Const isliye use kiya kyuki hum sirf data read kar rahe hain.

`return os`

Cascading statements allow karne ke liye.

Complete Program

```
#include <iostream>
using namespace std;

class Complex {

private:

    float real;
    float img;

public:

    Complex(float r = 0, float i = 0) {

        real = r;
        img = i;
    }

    friend ostream& operator<<(ostream& os, const Complex& c);
};

ostream& operator<<(ostream& os, const Complex& c) {

    os << "(" << c.real << ", " << c.img << ")";

    return os;
}

int main() {

    Complex c1(1.5, 2.5);
    Complex c2(4.2, 8.1);

    cout << c1 << endl;
    cout << c2;

    return 0;
}
```

Output

```
(1.5, 2.5)
(4.2, 8.1)
```

Cascading Example

```
cout << c1 << c2;
```

Actually:

```
operator<<( operator<<(cout, c1), c2 );
```

Real-Life Understanding

Socho:

```
cout
```

Ek pipeline hai.

Har << next data us pipeline mein bhejta jata hai.

19.4 Overloading Stream Extraction Operator (>>)

Ab hum user se Complex object ka input lena chahte hain.

Complex Class

```
class Complex {  
  
private:  
  
    float real;  
    float img;  
  
public:  
  
    friend istream& operator>>(istream& in, Complex& c);  
};
```

Stream Extraction Operator Code

```
istream& operator>>(istream& in, Complex& c) {  
  
    in >> c.real;  
    in >> c.img;  
  
    return in;  
}
```

Important Point

Yahan `Complex` object const nahi hoga.

Kyuki values change ho rahi hain.

Complete Program

```
#include <iostream>
using namespace std;

class Complex {

private:

    float real;
    float img;

public:

    friend istream& operator>>(istream& in, Complex& c);

    friend ostream& operator<<(ostream& os, const Complex& c);
};

istream& operator>>(istream& in, Complex& c) {

    in >> c.real;
    in >> c.img;

    return in;
}

ostream& operator<<(ostream& os, const Complex& c) {

    os << "(" << c.real << ", " << c.img << ")";

    return os;
}

int main() {

    Complex c1;

    cout << "Enter real and imaginary values: ";

    cin >> c1;

    cout << "Complex Number: " << c1;

    return 0;
}
```

Input

```
5.5 7.2
```

Output

```
Complex Number: (5.5, 7.2)
```

Real-Life Example

Socho:

```
cin >> c1;
```

Matlab:

"User se values lo aur Complex object mein bhar do."

Difference Between << and >>

Operator	Direction	Purpose
<<	Program → Screen	Output
>>	User → Program	Input

Why Stream Operators are Friend Functions?

Kyuki left side par:

```
cout
```

ya:

```
cin
```

hota hai.

Agar member function hota to left object `Complex` hona chahiye tha.

Lekin actual syntax:

```
cout << c1;
```

hai.

Isliye non-member friend function best choice hai.

Equality Operator (==)

Ye check karta hai ke do objects equal hain ya nahi.

Example

```
class Complex {  
  
private:  
  
    float real;  
    float img;  
  
public:  
  
    Complex(float r = 0, float i = 0) {  
  
        real = r;  
        img = i;  
    }  
  
    bool operator==(const Complex& c);  
};
```

Equality Operator Code

```
bool Complex::operator==(const Complex& c) {  
  
    if ((real == c.real) &&  
        (img == c.img)) {  
  
        return true;  
    }  
  
    else {  
  
        return false;  
    }  
}
```


Complete Example

```
#include <iostream>
using namespace std;

class Complex {

private:

    float real;
    float img;

public:

    Complex(float r = 0, float i = 0) {

        real = r;
        img = i;
    }

    bool operator==(const Complex& c);
};

bool Complex::operator==(const Complex& c) {

    if ((real == c.real) &&
        (img == c.img)) {

        return true;
    }

    return false;
}

int main() {

    Complex c1(2, 3);
    Complex c2(2, 3);

    if (c1 == c2) {

        cout << "Objects are equal";
    }

    else {
```

```
        cout << "Objects are not equal";  
    }  
  
    return 0;  
}
```

Output

```
Objects are equal
```

Inequality Operator (`!=`)

Ye check karta hai ke objects different hain ya nahi.

Code

```
bool Complex::operator!=(const Complex& c) {  
  
    if ((real != c.real) ||  
        (img != c.img)) {  
  
        return true;  
    }  
  
    return false;  
}
```

Complete Example

```
#include <iostream>
using namespace std;

class Complex {

private:

    float real;
    float img;

public:

    Complex(float r = 0, float i = 0) {

        real = r;
        img = i;
    }

    bool operator!=(const Complex& c);
};

bool Complex::operator!=(const Complex& c) {

    if ((real != c.real) ||
        (img != c.img)) {

        return true;
    }

    return false;
}

int main() {

    Complex c1(2, 3);
    Complex c2(5, 7);

    if (c1 != c2) {

        cout << "Objects are different";
    }

    else {
```

```
        cout << "Objects are same";  
    }  
  
    return 0;  
}
```

Output

Objects are different

Important Concepts Summary

Concept	Explanation
<<	Output operator
>>	Input operator
ostream	Output stream class
istream	Input stream class
Friend Function	Stream operators ke liye best
Cascading	Multiple operators ek line mein
return os	Cascading enable karta hai
const Complex&	Object sirf read karna
Complex&	Object modify karna

Real-Life Analogy

Programming Concept	Real-Life Example
<code>cout</code>	Delivery system
<code>cin</code>	Data collector
<code><<</code>	Parcel bhejna
<code>>></code>	Parcel receive karna
Stream	Pipe jisme data flow hota hai

Final Summary

Lecture 19 mein humne seekha:

- Stream insertion operator (`<<`)
- Stream extraction operator (`>>`)
- Operator overloading for user-defined classes
- Friend functions
- Cascading statements
- Equality (`==`) operator
- Inequality (`!=`) operator

Ye concepts C++ OOP mein bohat important hain aur real software applications mein extensively use hote hain.